

Homework — Week 3 coding (due before next class)

Instructor: fill due date __

Submit a GitHub repo (same as weeks 1-2, or `epic-go-week03`). **No zip files.**

```
week03/  
  go.mod  
  main.go           # optional demos / prints  
  homework/  
    homework.go     # your functions  
    homework_test.go # green tests  
  notes.md          # short answers (Q1-Q3 only)
```

```
cd week03  
go mod init epiclearn/week03 # once  
cd homework && go test -v
```

Use only Week 3 tools: `if` / `switch` / `for` / `range` , slices, maps, functions, `go test` .
No structs methods, no pointers theory, no HTTP, no SMS repo.

Part A — short notes (`notes.md`)

Answer in **2-4 sentences each** (your words):

1. Why must you write `s = append(s, x)` instead of only `append(s, x)` ?
2. After `b := a[:2]; b = append(b, 99)` , why can `a` change? How do you make an independent copy?
3. How do you tell “key missing” from “key present with value `0`” on a map?

Part B — coding (all in `homework/`)

Implement these functions. Keep them small. Add table-driven tests.

1. `contains(list []string, want string) bool`

Return true if `want` appears at least once.

list	want	result
<code>["SE", "CS"]</code>	<code>"SE"</code>	true
<code>["SE", "CS"]</code>	<code>"IT"</code>	false
<code>nil</code>	<code>"SE"</code>	false

2. Unique(list []string) []string

First occurrence wins; **keep order**.

input	want
<code>["SE", "CS", "SE", "IT", "CS"]</code>	<code>["SE", "CS", "IT"]</code>
<code>nil</code> / empty	empty slice (len 0)

3. Freq(words []string) map[string]int

Count how often each word appears.

input	want
<code>["a", "b", "a"]</code>	<code>a:2, b:1</code>
<code>empty</code> / <code>nil</code>	<code>empty map (len == 0)</code>

Assert with **comma-ok**. Do not compare `fmt.Sprintf(map)`.

4. Filter(list []string, keep string) []string

Return a new slice with only values equal to `keep` (SMS preview: filter by course).

list	keep	want
<code>["SE", "CS", "SE"]</code>	<code>"SE"</code>	<code>["SE", "SE"]</code>
<code>["SE", "CS"]</code>	<code>"IT"</code>	empty

5. Max(nums []int) (int, bool)

Return the largest value. Second result is `false` if the slice is empty/nil (no panic).

nums	want	ok
<code>[]int{3, 9, 4}</code>	9	true
nil / empty	0	false

6. `Reverse(nums []int) []int`

Return a **new** reversed slice. Do not mutate the caller's slice (backing-array lesson).

input	want
<code>[]int{1, 2, 3}</code>	<code>[]int{3, 2, 1}</code>
after call, original still	<code>[]int{1, 2, 3}</code>

7. `Intersection(a, b []string) []string`

Values that appear in **both** slices. Order = first-seen order from `a`. No duplicates in the result.

a	b	want
<code>["SE", "CS", "IT"]</code>	<code>["IT", "SE", "SE"]</code>	<code>["SE", "IT"]</code>
<code>["SE"]</code>	<code>["CS"]</code>	empty

Hint: build a `map[string]bool` from `b`, then range `a`.

8. `Grade(score int) string (light algorithm + switch)`

score	grade
75-100	"A"
65-74	"B"
50-64	"C"
0-49	"F"
anything else	"?"

Use a **tagless** `switch`.

Skeleton (optional start)

```
package homework

func Contains(list []string, want string) bool { /* ... */ }

func Unique(list []string) []string { /* ... */ }

func Freq(words []string) map[string]int { /* ... */ }

func Filter(list []string, keep string) []string { /* ... */ }

func Max(nums []int) (int, bool) { /* ... */ }

func Reverse(nums []int) []int { /* ... */ }

func Intersection(a, b []string) []string { /* ... */ }

func Grade(score int) string { /* ... */ }
```

Stretch (optional, not required to pass)

Pick **one**:

- `SumEven(nums []int) int` — sum only even numbers
- `IsPalindrome(s string) bool` — ignore spaces; compare runes, not bytes
- `TopWord(words []string) (string, int)` — word with highest frequency (if tie, any is fine)

Add tests if you do a stretch.

How it is marked

Requirement	Pass
Repo URL sent	Yes
<code>notes.md</code> answers Q1–Q3	Your words
<code>go test</code> in <code>homework/</code> is green for functions 1–8	Required
<code>gofmt</code> clean, no binaries committed	Yes

Requirement	Pass
Stretch	Bonus only

Time box: aim for **2-4 hours**. If stuck >20 minutes on one function, move on and ask next class — bring your failing test output.